



UNIVERSITY OF
COPENHAGEN

Transformation and Rejection Sampling

Computational Statistics

Johan Larsson

Department of Mathematical Sciences, University of Copenhagen

September 17, 2026

Transformation Methods

Transform samples from a simpler distribution, including by inversion of the CDF.

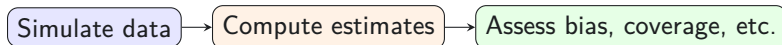
Rejection Sampling

Sample using a density that may be known only up to normalization.

Why Do We Need a Sampler?

Simulation lets us explore the consequences of a statistical model when analytic calculations are difficult.

How biased is an estimator in a small sample? Does a nominal 95% confidence interval actually achieve 95% coverage?



Draws also let us estimate probabilities and expectations, including tail probabilities and posterior summaries.

All of these applications need **reliable and efficient sampling from a specified distribution**.

Pseudo-Random Numbers

The basis for all sampling methods is a source of random numbers and R uses deterministic algorithms to generate **pseudo-random** numbers.

The sequences are reproducible from a seed but designed to appear random.

A research field in itself. See ?RNG in R for available algorithms.

Mersenne Twister

The default in R, which generates integers in the range

$$\{0, 1, \dots, 2^{32} - 1\}.$$

Long period: $2^{19937} - 1$. Blocks of up to 623 consecutive integers occur nearly equally often over a full period.

Uniform Random Variables in R

`runif()` scales these pseudo-random integers to the unit interval, with adjustments to exclude 0 and 1.

From Uniform Random Numbers to Target Distributions

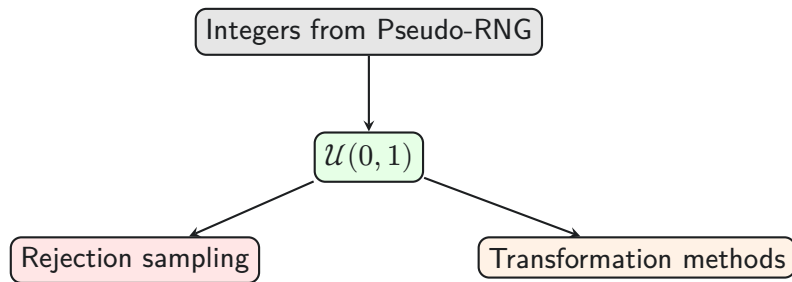
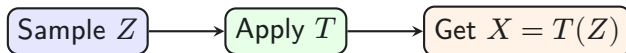


Figure 1: Main categories of sampling methods (in this course).

To sample from a target distribution with CDF F , choose a random variable Z that we can simulate and a computable, measurable transformation T such that

$$\Pr(T(Z) \leq x) = F(x), \quad x \in \mathbb{R}.$$



Example

To sample $X \sim \text{Exp}(\lambda)$ with rate $\lambda > 0$, use $X = -\log(U)/\lambda$ with $U \sim \mathcal{U}(0, 1)$.

For $x \geq 0$,

$$\Pr\left(-\frac{\log U}{\lambda} \leq x\right) = \Pr(U \geq e^{-\lambda x}) = 1 - e^{-\lambda x}.$$

Example: Sampling from a t -Distribution

If we let

$$T(z, w) = \frac{z}{\sqrt{w/k}},$$

and take independent $Z \sim \mathcal{N}(0, 1)$ and $W \sim \chi_k^2$ with $k > 0$, then

$$X = T(Z, W) = \frac{Z}{\sqrt{W/k}} \sim t_k.$$

This transformation is used in R and other software to simulate t -distributed random variables.

Algorithm 1: Transformation sampling for the t -distribution

$z \leftarrow$ sample from $\mathcal{N}(0, 1)$;

$w \leftarrow$ sample from χ_k^2 , independently of z ;

return $z/\sqrt{w/k}$;

A type of **transformation** sampling.

If $F^{-1} : (0, 1) \rightarrow \mathbb{R}$ is the generalized inverse of a distribution function F and U is uniformly distributed on $(0, 1)$, then

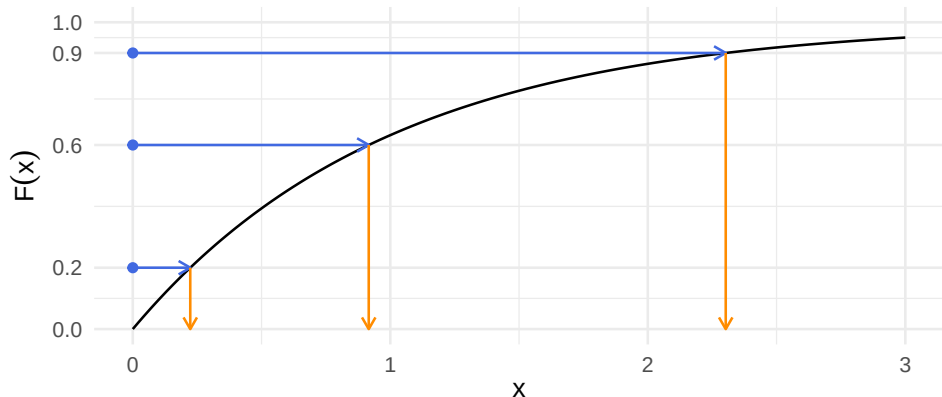
$$F^{-1}(U)$$

has distribution function F .

Computing F^{-1} can be simple (e.g., for the exponential distribution) or require a search or numerical approximation.

Following the Inverse CDF

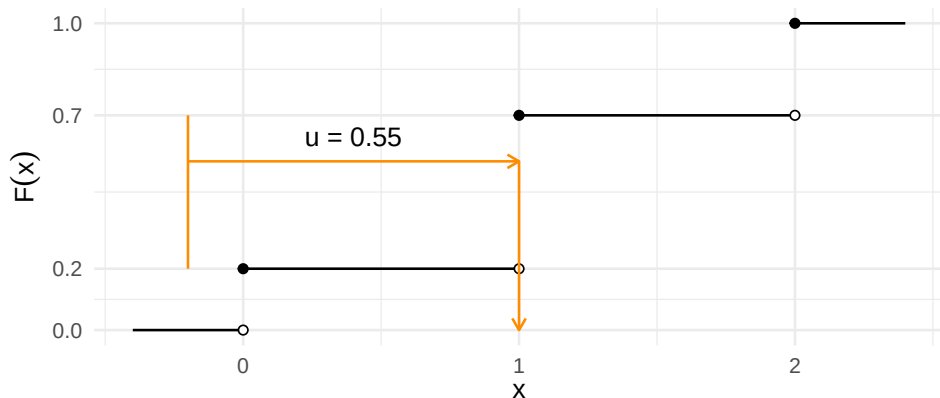
For $X \sim \text{Exp}(1)$, $F(x) = 1 - e^{-x}$ for $x \geq 0$.



Move right from U to the CDF, then down to $X = F^{-1}(U) = -\log(1 - U)$.

Inversion for a Discrete Distribution

The **generalized inverse** is $F^{-1}(u) = \inf\{x : F(x) \geq u\}$.



Every $u \in (0.2, 0.7]$ maps to 1, so $\Pr(X = 1) = 0.7 - 0.2 = 0.5$.

Von Mises Distribution

The density on $(-\pi, \pi]$ is

$$f(x) = \frac{1}{2\pi I_0(\kappa)} e^{\kappa \cos(x-\mu)},$$

where $\mu \in (-\pi, \pi]$ is the location, $\kappa \geq 0$ is the concentration, and I_0 is the modified Bessel function of the first kind of order 0.

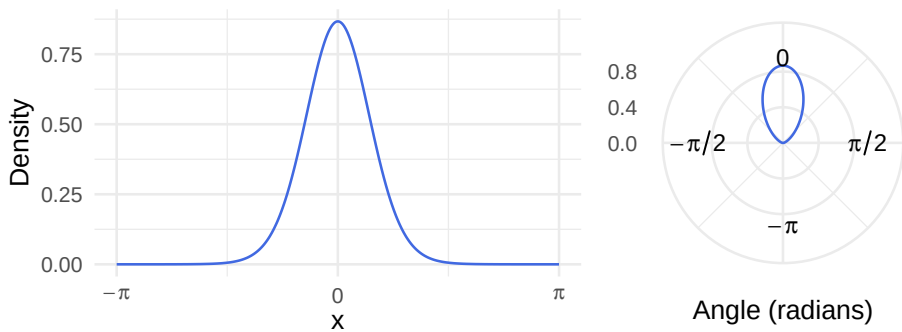


Figure 2: The von Mises density for $\mu = 0$ and $\kappa = 5$.

Inverting the von Mises CDF

For the same distribution with $\mu = 0$ and $\kappa = 5$,

$$F(x) = \int_{-\pi}^x f(t) dt, \quad x \in [-\pi, \pi].$$

We can evaluate F by numerically integrating the density `dvm`:

```
pvm <- function(x, kappa = 5) {  
  integrate(  
    function(t) dvm(t, kappa),  
    lower = -pi,  
    upper = x,  
    rel.tol = 1e-10  
  )$value  
}
```

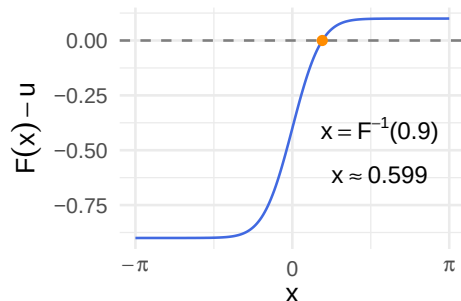
There is no elementary inverse CDF. We solve $F(x) - u = 0$ numerically.

Sampling by Solving $F(x) = U$

To sample from this von Mises distribution, draw $U \sim \mathcal{U}(0, 1)$, then solve $F(x) - U = 0$.

For one draw, suppose $u = 0.9$.

```
u <- 0.9
sample_x <- uniroot(
  function(x) pvm(x) - u,
  interval = c(-pi, pi),
  tol = 1e-8
)$root
sample_x
```



```
[1] 0.5994823
```

Each new draw requires another solve, with numerical integration at each CDF evaluation.

Sampling Directly from the von Mises Density

We can now sample by numerical inversion. Evaluating the unnormalized density

$$q(x) = e^{\kappa \cos(x-\mu)}$$

is much simpler than computing the CDF.

Sampling Problem

Can we sample using q directly, without computing or inverting the CDF?

Yes! We can use **rejection sampling**.

We want to sample from a target density f using a proposal density g that we can sample from easily.

Let $Y_1, Y_2, \dots$ be i.i.d. with density g on $\mathbb{R}$ and $U_1, U_2, \dots$ be i.i.d. $\mathcal{U}(0, 1)$, with the two sequences independent.

Define

$$\sigma = \inf \left\{ n \geq 1 \mid U_n \leq \alpha \frac{f(Y_n)}{g(Y_n)} \right\},$$

for $\alpha \in (0, 1]$, where f is a density.

Theorem

If $f(y) \leq g(y)/\alpha$ for all $y \in \mathbb{R}$, then the distribution of Y_σ has density f .

α is the **acceptance probability** and $g(y)/\alpha$ the **envelope** of f .

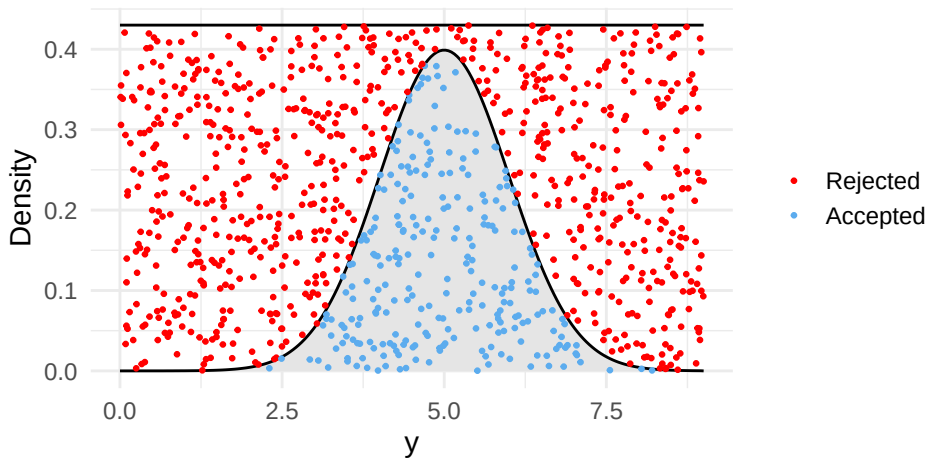


Figure 3: Rejection sampling for a normal target truncated to $(0, 9)$, using a uniform proposal. Points have height $Ug(Y)/\alpha$.

Why Rejection Sampling Works

For one proposal $Y \sim g$, let $A = \{U \leq \alpha f(Y)/g(Y)\}$ be the acceptance event.

Writing F for the target CDF,

$$\Pr(Y \leq x, A) = \int_{-\infty}^x g(y) \frac{\alpha f(y)}{g(y)} dy = \alpha F(x).$$

Taking $x \rightarrow \infty$ gives $\Pr(A) = \alpha$. Therefore,

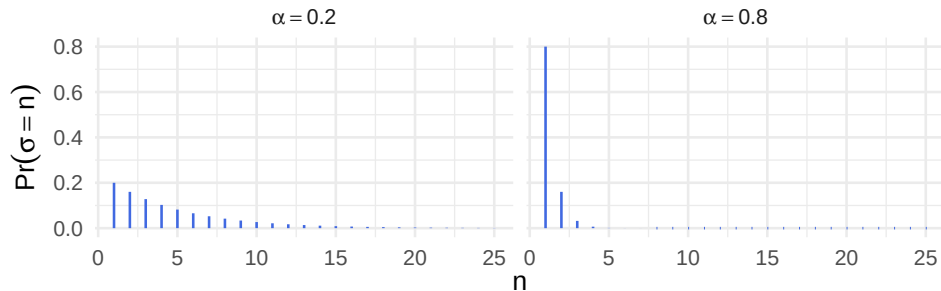
$$\Pr(Y \leq x \mid A) = \frac{\Pr(Y \leq x, A)}{\Pr(A)} = \frac{\alpha F(x)}{\alpha} = F(x).$$

Each accepted proposal has the target distribution. Fresh independent attempts give the first accepted proposal, Y_σ , this same distribution.

The Number of Proposals

Each attempt succeeds with probability α , independently:

$$\Pr(\sigma = n) = (1 - \alpha)^{n-1}\alpha, \quad n = 1, 2, \dots$$



σ counts attempts **including the accepted one**: $\mathbb{E}[\sigma] = 1/\alpha$.

Stopping at the N th acceptance takes N/α proposals on average.

Normalizing Constants

Rejection sampling works with unnormalized densities!

Suppose:

- Target: $f(y) = cq(y)$ (unknown c)
- Proposal: $g(y) = dp(y)$ (can sample from g , unknown d)

Acceptance test:

$$U \leq \frac{\alpha f(y)}{g(y)} = \frac{\alpha cq(y)}{dp(y)}$$

If $\alpha = \frac{\alpha' d}{c}$, then

$$\frac{\alpha f(y)}{g(y)} = \frac{\alpha' q(y)}{p(y)}$$

Takeaway

You do not need to know c or d !

How to Find α' ?

Given functions q and p , we need to find α' such that

$$\alpha' q(y) \leq p(y) \quad \text{for all } y.$$

How?

We can try to solve the problem

$$\alpha'_{\max} = \inf_{y: q(y) > 0} \frac{p(y)}{q(y)}.$$

We need $0 < \alpha' \leq \alpha'_{\max}$.

An analytic bound guarantees validity.

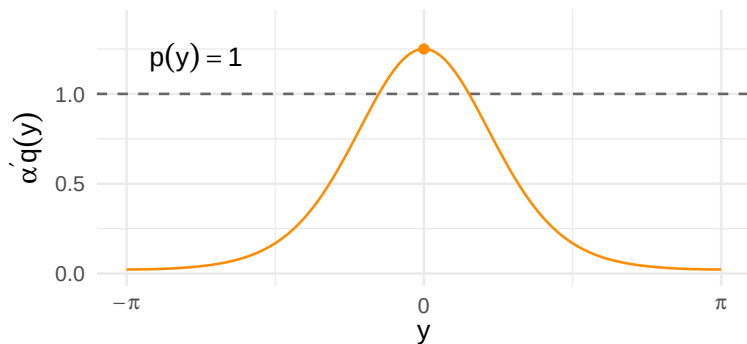
Numerical optimization (e.g., `optimize()` in R) can suggest a bound, but the envelope inequality must hold everywhere.

Finding the von Mises Bound

For $\mu = 0$, take $q(y) = e^{\kappa \cos y}$ and $p(y) = 1$ on $(-\pi, \pi)$.

Invalid

$$\kappa = 2, \alpha' = 1.25e^{-\kappa}$$



Since $\cos y \leq 1$, $\alpha'_{\max} = \inf_y e^{-\kappa \cos y} = e^{-\kappa}$, attained at $y = 0$.

Von Mises Rejection Sampling

For $\mu = 0$, use the uniform proposal on $(-\pi, \pi)$: $g(y) = 1/(2\pi)$.

Since

$$e^{\kappa(\cos(y)-1)} = \alpha' e^{\kappa \cos(y)} \leq 1,$$

where $\alpha' = \exp(-\kappa)$, we reject if

$$U > e^{\kappa(\cos(Y)-1)}.$$

Algorithm 2: Rejection sampling for the von Mises distribution

repeat

 Generate $Y \sim \mathcal{U}(-\pi, \pi)$;

 Generate $U \sim \mathcal{U}(0, 1)$;

if $U \leq e^{\kappa(\cos(Y)-1)}$ **then**

 Accept Y ;

until *sample is accepted*;

Compute the Acceptance Test on the Log Scale

The same rejection test can be written as

$$\log U \leq \log \alpha' + \log q(Y) - \log p(Y).$$

For the posterior in Assignment 2A, compute the log-density as a sum:

```
log_q <- function(theta, x, z) {  
  if (theta < 0) {  
    return(-Inf)  
  }  
  eta <- theta * x  
  sum(z * eta - exp(eta))  
}
```

Multiplying many small likelihood contributions can underflow to zero. Work with log-densities and subtract them before exponentiating, if needed.

For von Mises sampling, the log acceptance ratio is simply $\kappa(\cos Y - 1)$.

Pitfalls: Missing Support

Take a standard normal target f and a $\mathcal{U}(-2, 2)$ proposal g .

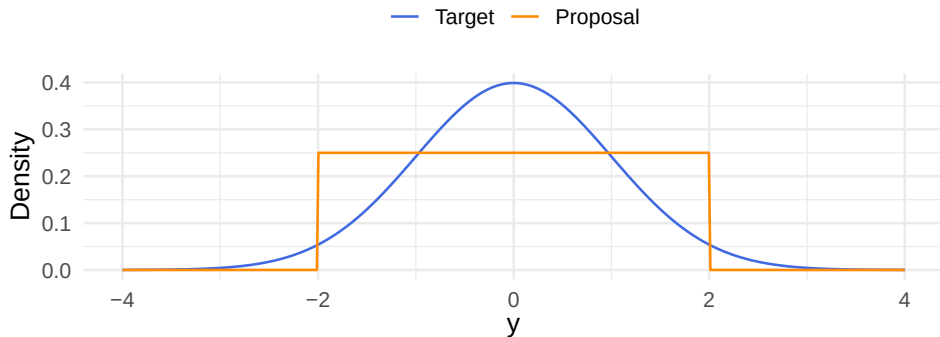


Figure 4: The uniform proposal cannot cover the normal target. No finite multiple of the proposal can envelope the target.

Here $g(y) = 0$ for $|y| > 2$, where $f(y) > 0$. No finite multiple of g can cover f .

Pitfalls: Tails Too Light

Take a standard normal target f and a $\mathcal{N}(0, 1/4)$ proposal g . Both densities are positive on $\mathbb{R}$.

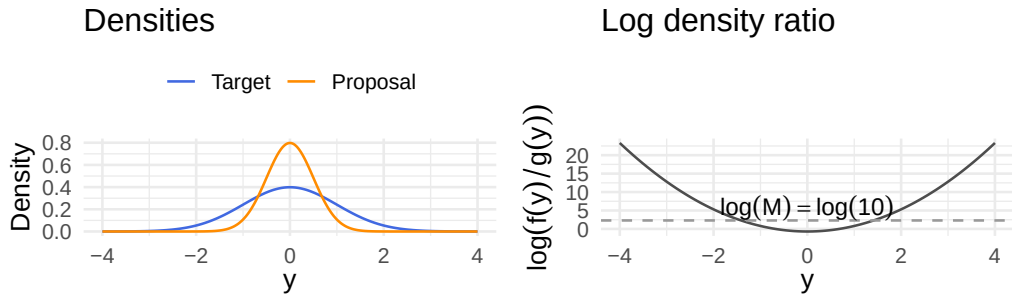


Figure 5: The normal proposal has lighter tails than the target. No finite multiple of the proposal can envelope the target.

$$\log \frac{f(y)}{g(y)} = -\log 2 + \frac{3}{2}y^2 \rightarrow \infty \quad \text{as } |y| \rightarrow \infty.$$

No finite M satisfies $f(y) \leq M g(y)$ for every y .

Implementing the von Mises Sampler

```
get_one_sample_vmises <- function(kappa) {  
  repeat {  
    Y <- runif(1, -pi, pi)  
    U <- runif(1)  
    if (log(U) <= kappa * (cos(Y) - 1)) {  
      break  
    }  
  }  
  Y  
}
```

```
sample_vonmises_slow <- function(N, kappa) {  
  replicate(N, get_one_sample_vmises(kappa))  
}
```

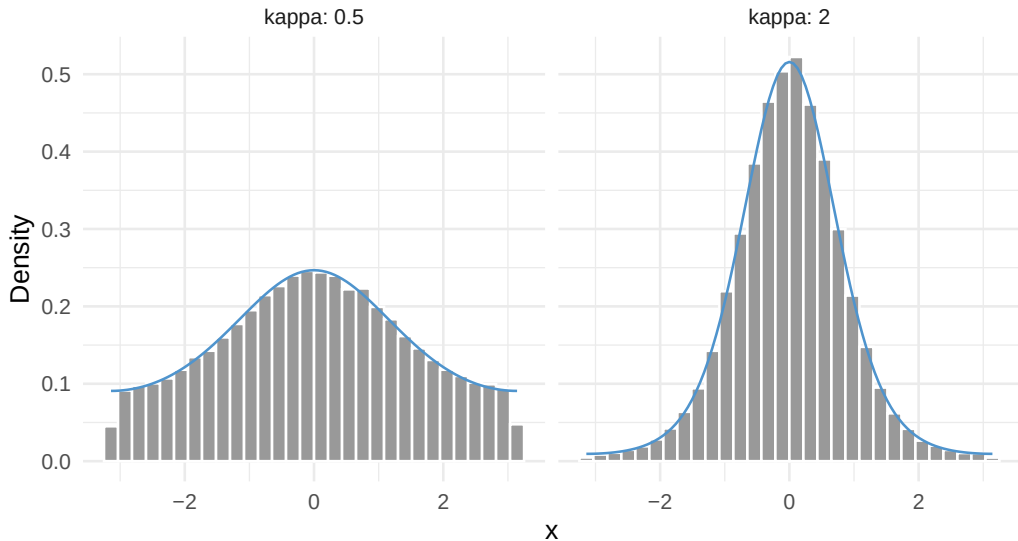


Figure 6: Histogram of samples from `sample_vonmises_slow()`, with theoretical density overlaid.

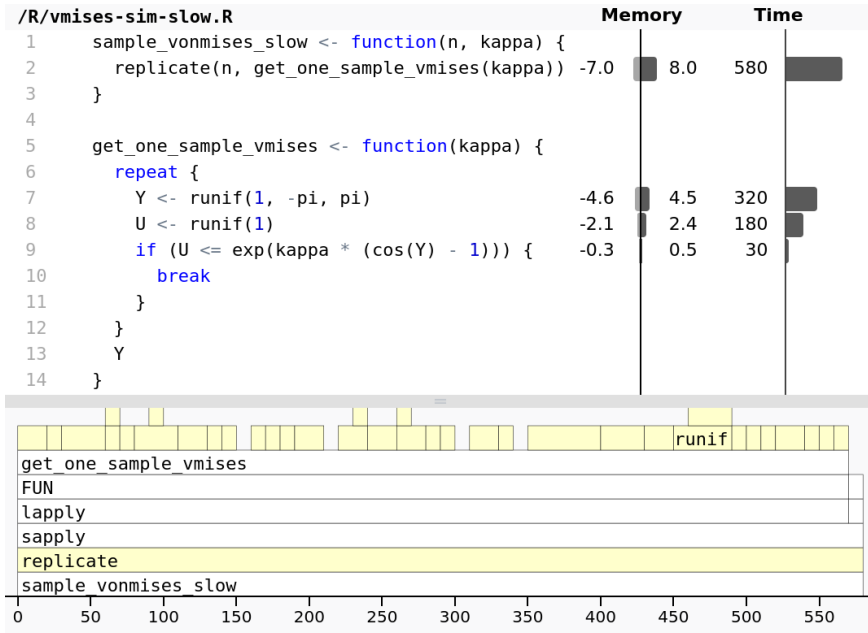


Figure 7: A line profile of the naive von Mises sampling implementation.

The bottleneck is the repeated calls to `runif(1)`.

Both loop overhead and calling the RNG for one number at a time add cost.

It is faster to generate random numbers in batches and store them in vectors.

But how can we do that for rejection sampling with an **unknown** number of rejections?

Vectorized (but Random Length) Rejection Sampler

We can speed up the rejection sampler by generating m proposal samples at once.

```
fixed_vonmises_sampler <- function(m, kappa) {  
  Y <- runif(m, -pi, pi)  
  u <- runif(m)  
  accept <- log(u) <= kappa * (cos(Y) - 1)  
  Y[accept]  
}
```

Simple and fast, but returns a **random** number of samples!

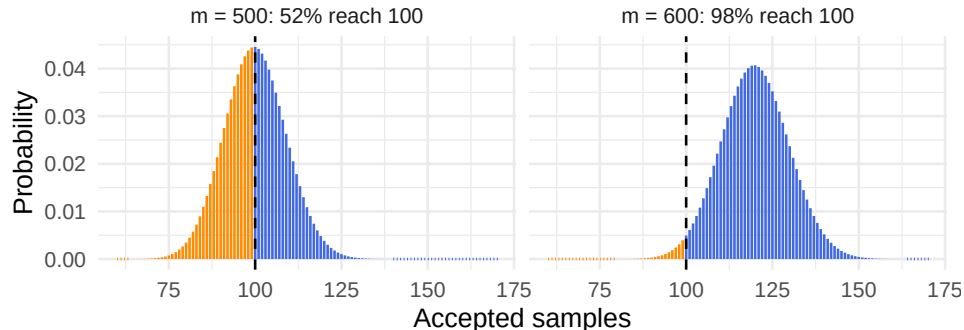
We need a function that returns exactly N samples. We can estimate α to choose batch sizes without computing it in advance.

How Many Samples Does a Batch Return?

We want **100 accepted samples**. How many proposals should we generate?

Suppose $\alpha = 0.2$. From m independent proposals, the accepted count is

$$B_m \sim \text{Binomial}(m, \alpha), \quad \mathbb{E}[B_m] = m\alpha.$$



$m = 100/0.2 = 500$ gives 100 accepted samples **on average**. If a batch falls short, generate another batch.

Algorithm 3: Sampling exactly N values via rejection sampling with unknown acceptance probability α .

```
 $\hat{\alpha} \leftarrow 1;$   
 $\mathbf{y} \leftarrow$  empty vector;  
 $n_{\text{accepted}} \leftarrow 0;$   
while  $n_{\text{accepted}} < N$  do  
     $m \leftarrow \lceil (N - n_{\text{accepted}}) / \hat{\alpha} \rceil;$   
    Generate  $m$  proposals and apply the rejection test;  
     $\mathbf{y}_{\text{new}} \leftarrow$  accepted proposals;  
     $n_{\text{accepted}} \leftarrow n_{\text{accepted}} + \text{length}(\mathbf{y}_{\text{new}});$   
    if  $\mathbf{y}$  is empty then  
         $\hat{\alpha} \leftarrow (n_{\text{accepted}} + 1) / (m + 1)$            // Keep the estimate positive.  
    Append  $\mathbf{y}_{\text{new}}$  to  $\mathbf{y};$   
return  $\mathbf{y}_{1:N}$ 
```

Sampling with movMF

We can compare our samplers with the implementation in movMF.

For $\mu = 0$, use the parameter vector $(\kappa, 0)$. The package returns points on the unit circle, which we convert to angles.

```
library(movMF)

sample_vonmises_movMF <- function(N, kappa) {
  xy <- rmovMF(N, c(kappa, 0))
  # atan2 preserves the quadrant of each point on the unit circle.
  atan2(xy[, 2], xy[, 1])
}

x <- sample_vonmises_movMF(500, 5)
```

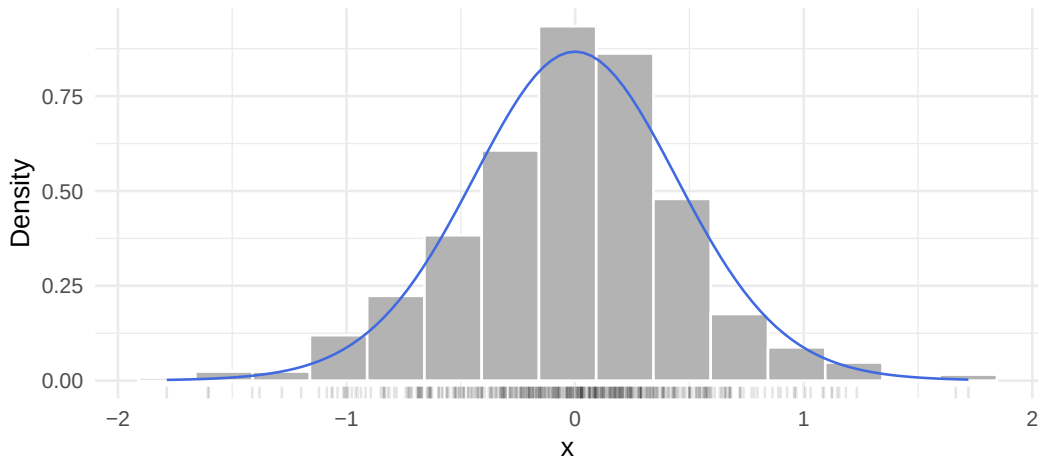


Figure 8: Histogram of samples from `rmovMF()` for $\mu = 0$ and $\kappa = 5$, with the theoretical density in blue.

Check the von Mises Samplers Before Timing

Each sampler should reproduce the von Mises CDF with $\mu = 0$ and $\kappa = 5$.

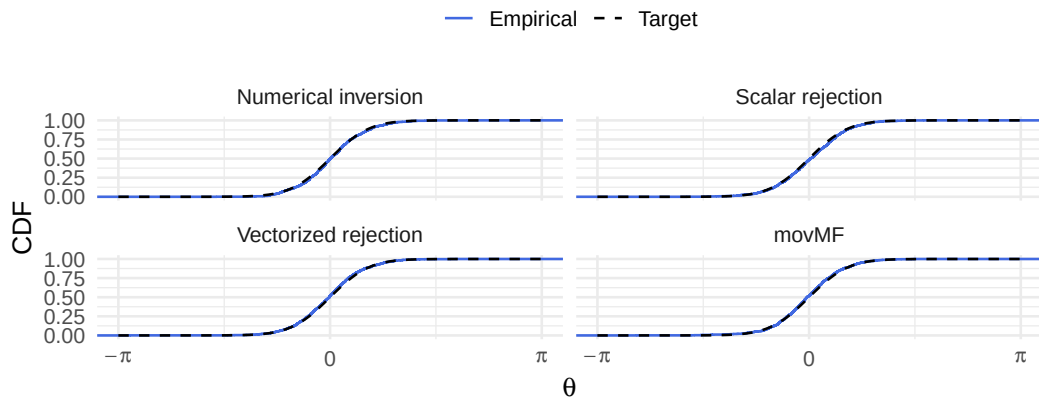


Figure 9: $N = 1000$ draws per method. The target CDF is evaluated by numerical integration.

The empirical CDFs agree with the target. Next, compare the time per draw.

Benchmarking the von Mises Samplers

Each method returns $N = 1000$ angles with $\mu = 0$ and $\kappa = 5$.

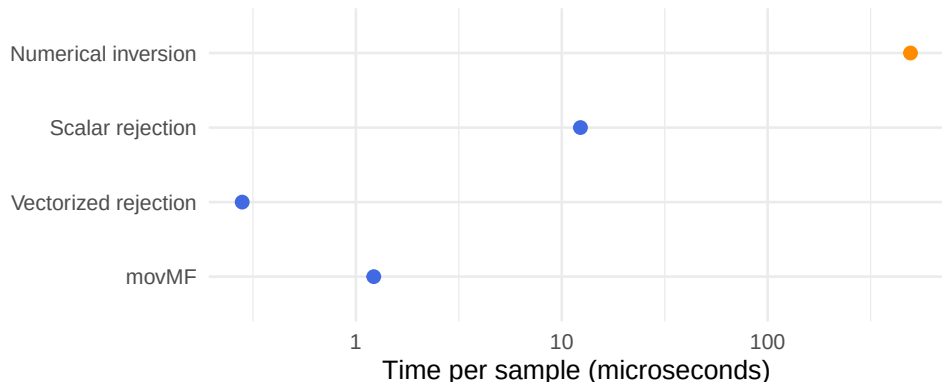


Figure 10: Median time per returned sample over five runs, on a logarithmic scale.

Repeated integration and root finding make this direct inversion sampler expensive.

Piecewise Exponential Envelopes

When f is positive and *log-concave* on an interval I , we can construct an integrable envelope of the form

$$f(x) \leq e^{V(x)}$$

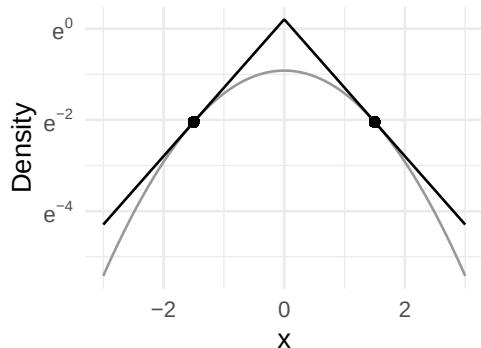
where

$$V(x) = \sum_{i=1}^m (a_i x + b_i) \mathbf{1}_{I_i}(x)$$

for intervals I_i forming a partition of I .

Typically, $a_i x + b_i$ is tangent to the graph of $\log(f)$ at $x_i \in I_i = (z_{i-1}, z_i]$ for

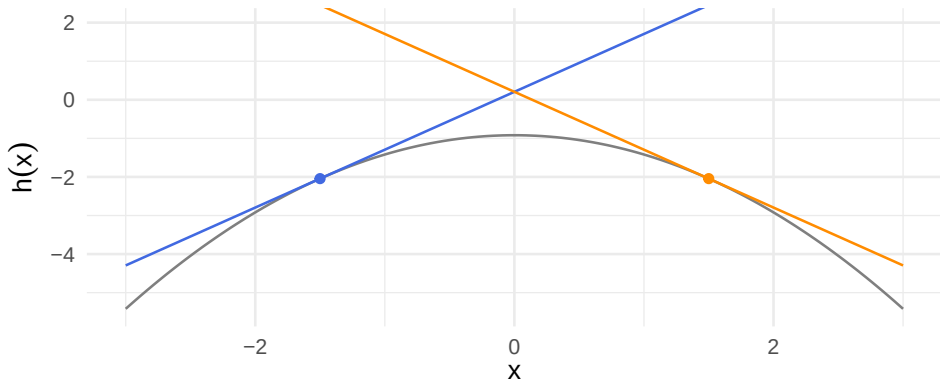
$$z_0 < x_1 < z_1 < x_2 < \dots < z_{m-1} < x_m < z_m.$$



Why Tangents Give Upper Bounds

If $h = \log f$ is differentiable and concave, then

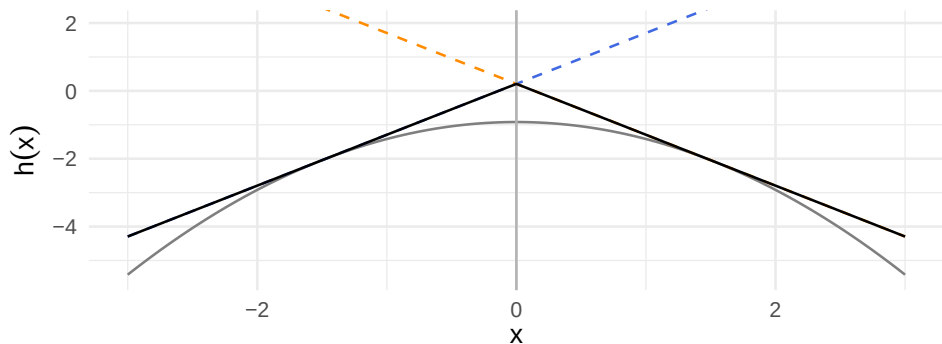
$$h(x) \leq h(x_i) + h'(x_i)(x - x_i) = a_i x + b_i.$$



Choose $a_i = h'(x_i)$ and $b_i = h(x_i) - a_i x_i$. Exponentiating gives $f(x) \leq e^{a_i x + b_i}$.

Where Should the Pieces Meet?

Use the smallest tangent bound at each point: $V(x) = \min_i \{a_i x + b_i\}$.



Adjacent tangents meet at

$$z_i = \frac{b_{i+1} - b_i}{a_i - a_{i+1}} \quad (a_i \neq a_{i+1}).$$

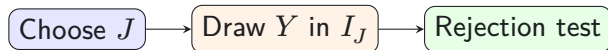
Here $z_1 = 0$. Keep the blue tangent to the left and the orange tangent to the right.

Sampling from the Envelope

For each piece $I_i = (L_i, R_i]$, compute its finite mass

$$w_i = \int_{L_i}^{R_i} e^{a_i x + b_i} dx, \quad C = \sum_i w_i.$$

Choose a piece J with $\Pr(J = i) = w_i/C$.



Given $J = i$ and $a_i \neq 0$, draw $U \sim \mathcal{U}(0, 1)$ and set

$$Y = \frac{1}{a_i} \log[(1 - U)e^{a_i L_i} + Ue^{a_i R_i}].$$

Use endpoint limits for infinite bounds. If $a_i = 0$, sample uniformly on the finite interval.

This gives $g(y) = e^{V(y)}/C$. Accept with probability $f(y)/e^{V(y)}$.

Beta Distribution

For shape parameters $\alpha, \beta > 1$, the beta density is log-concave.

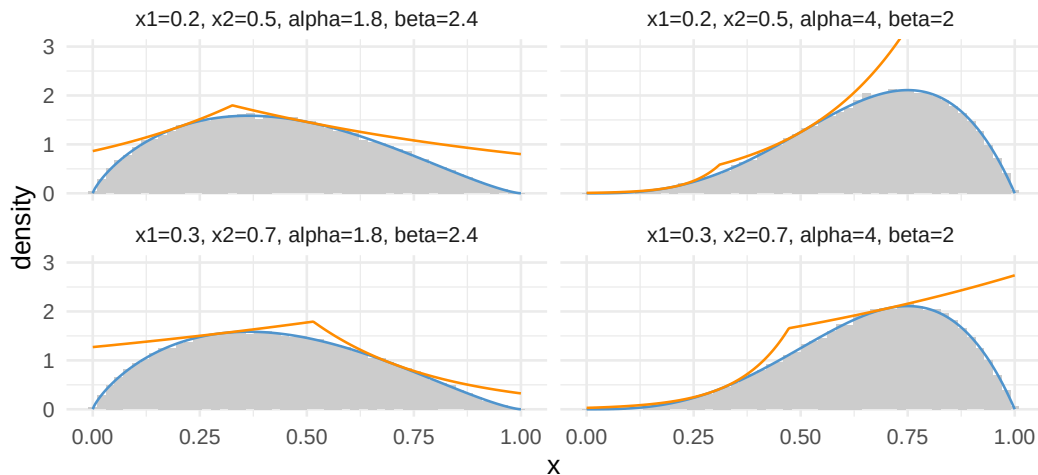


Figure 11: Samples from a piecewise exponential envelope sampler, with densities (blue) and envelopes (orange) overlaid.

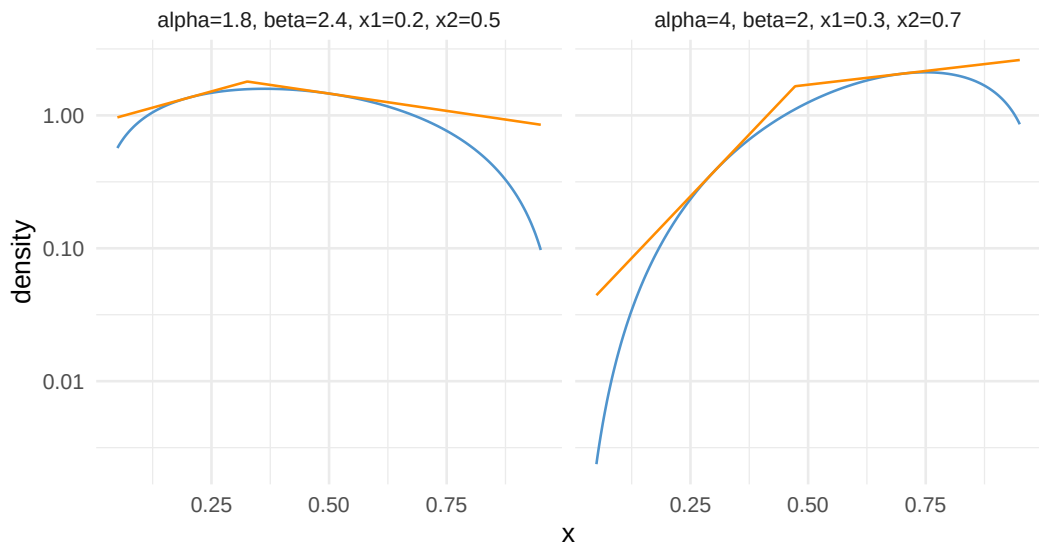


Figure 12: Theoretical densities (blue) and envelopes (orange) for different parameter values and choices of x_1 and x_2 .

The examples above use fixed envelopes. Adaptive rejection sampling updates the envelope using new points.

For a fixed envelope, a simple cost model is

$$\mathbb{E}[T] \approx T_{\text{setup}} + \frac{N}{\alpha} c_{\text{attempt}},$$

where c_{attempt} includes generating and testing a proposal.

Batching reduces overhead per proposal. A tighter envelope reduces rejections, but constructing, sampling, and updating it also cost time.

One Adaptive Update

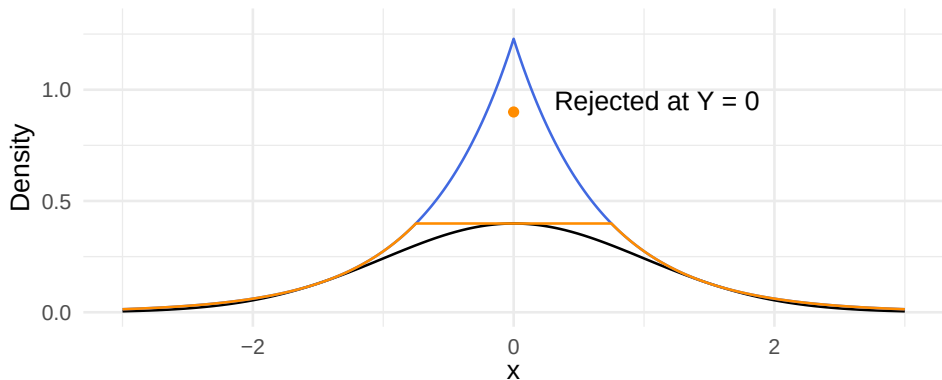
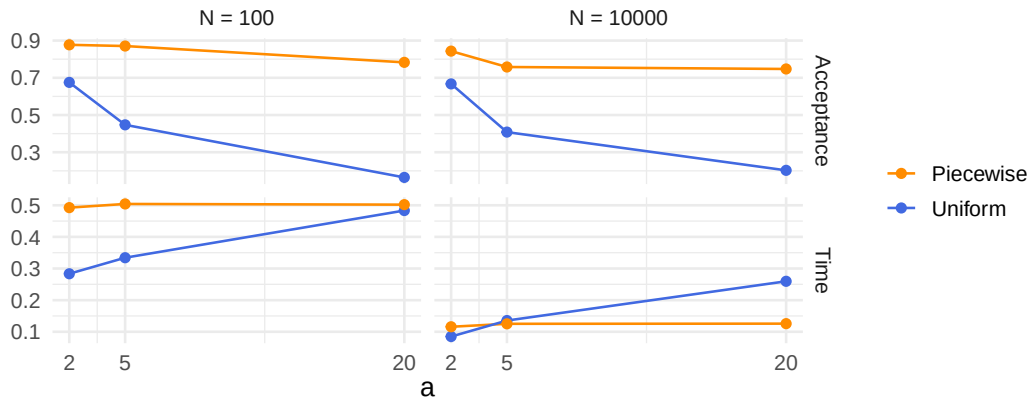


Figure 13: For a log-concave target, use a rejected proposal Y as a new tangent point.

$$V_{\text{new}}(x) = \min\{V_{\text{old}}(x), h(Y) + h'(Y)(x - Y)\}.$$

The orange envelope is tighter. Update the piece masses before the next draw.

Does a Tighter Envelope Run Faster?



Time is median microseconds per returned sample, including setup and all batches.
Acceptance is accepted/proposed, including surplus draws.

Transformation Methods vs. Rejection Sampling

Transformation Methods

Transform samples from a simple distribution to the target distribution.

Efficient if the transformation is available and cheap to compute.

Finding or evaluating a transformation can be difficult for complex or unnormalized distributions.

Takeaway

Use a simple transformation when available. Otherwise, consider rejection sampling with a suitable envelope.

Rejection Sampling

Uses a proposal distribution and accepts/rejects samples.

Works even if the quantile function is unavailable or expensive.

Efficiency depends on the proposal and acceptance probability.

Works with unnormalized densities, but acceptance can be very low in high dimensions.

Exercise 1

For a positive integer k , use the fact that

$$W = X_1^2 + \dots + X_k^2 \sim \chi_k^2 \quad \text{for} \quad X_1, \dots, X_k \sim_{\text{i.i.d.}} \mathcal{N}(0, 1)$$

to implement a function, `my_rchisq()`, such that `my_rchisq(N, k)` returns a sample of N i.i.d. observations from χ_k^2 .

Exercise 2

Make another function, `my_other_rchisq()`, that uses inversion sampling. Check the output distributions, then benchmark your implementations against one another and `rchisq()`.

Transformation Methods

- Transform samples from a simpler distribution to the target distribution.
- Inversion uses the target's quantile function to transform uniform samples.
- Efficient if the transformation is available and cheap to compute.

Rejection Sampling

- Use a proposal distribution and accept/reject samples.
- Works even if the quantile function is unavailable or expensive.

Monte Carlo Integration

Sample from a distribution to estimate expectations of functions of the random variable.

Importance Sampling

Weight samples from a proposal distribution to estimate expectations with respect to a target distribution.

Appendix

Von Mises Envelopes

For $\mu = 0$ and $\kappa > 0$, the von Mises density is not globally log-concave.

Since $(\log f)''(x) = -\kappa \cos(x)$, it is

- log-concave on $(-\pi/2, \pi/2)$ and
- log-convex on $(-\pi, -\pi/2)$ and $(\pi/2, \pi)$.

Use tangents in the center and chords in the tails to bound $\log f$.

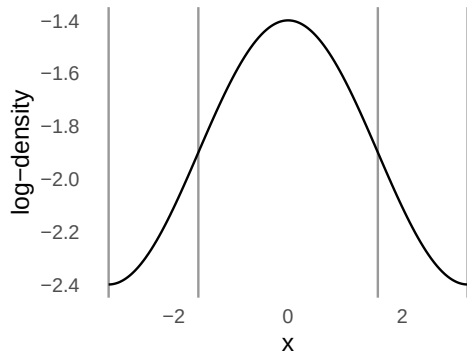


Figure 14: The von Mises log-density with $\mu = 0$ and $\kappa = 0.5$.

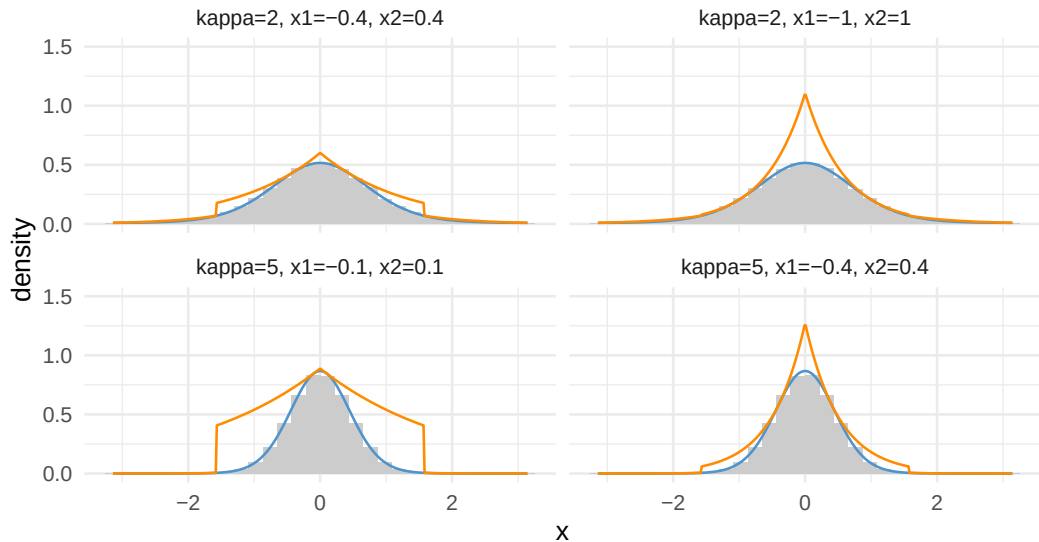
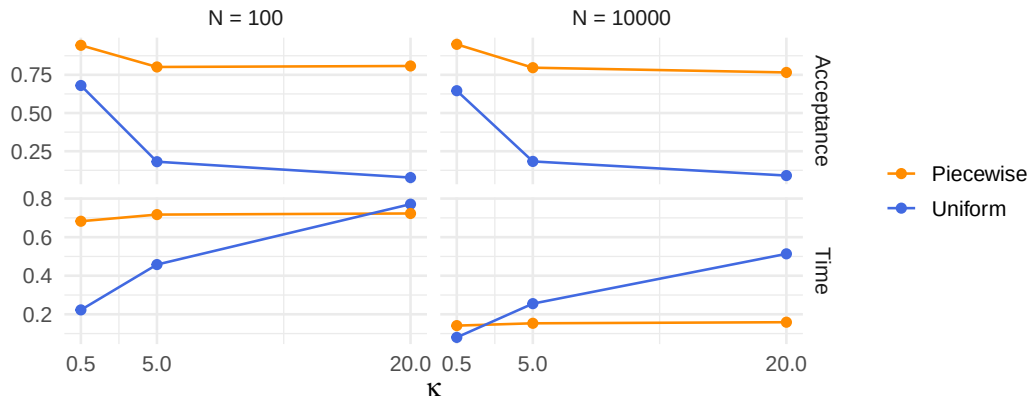


Figure 15: Von Mises samples from a piecewise exponential envelope sampler, with densities (blue) and envelopes (orange) overlaid.

Comparing the von Mises Envelopes

Compare the uniform proposal with the four-piece envelope. Use central tangent points $\pm \min(1, \kappa^{-1/2})$ to follow the target's concentration.



These envelopes stay fixed during sampling. Time is median microseconds per returned sample, including setup and all batches. Compare both rows as N and κ change.